

Aufgabenblatt 11

Übung zu Einführung in die Bildverarbeitung

Hanna Beitz, Malin Haas, Lasse Huber-Saffer, Tim Radke,
Teodora Rogozenco, Jannis Scheff, Christian Wilms
SoSe 2026

Ausgabe: 3. Juli 2025 - **Abgabe bis: 10. Juli 2026, 10:00 per Moodle!**

Dies ist das letzte Aufgabenblatt!

Lernziele

- Anwendung der Formeln zu den morphologischen Operationen vertiefen
- Geeignete morphologische Operationen auswählen können
- Vorteil von don't care Elementen verstehen

Aufgabe 1 — Erosion und Dilatation berechnen - 10+10+10=30 Punkte - Theorieaufgabe

Gegeben sei der Vordergrund A eines Binärbildes als Koordinatenmenge $A = \{(1, 1), (1, 2), (2, 2), (3, 2), (3, 3), (4, 1), (4, 2), (5, 2), (5, 3)\}$ und das Strukturelement B mit der Koordinatenmenge $B = \{(0, 0), (0, 1)\}$.

1. Berechnet die Erosion $A \ominus B$ mittels der aus den Folien bekannten Formel. Argumentiert dabei nicht über eine visuelle Darstellung von A und B , wie wir es in der Präsenzübung gemacht haben. Beschreibt auch kurz euer Vorgehen.
2. Berechnet analog die Dilatation $A \oplus B$ mittels der aus den Folien bekannten Formel. Argumentiert auch hier wieder nicht über eine visuelle Darstellung von A und B und beschreibt erneut kurz euer Vorgehen.
3. Wie verändern sich die Ergebnisse jeweils, wenn das Strukturelement B nur noch aus der Koordinatenmenge $\{(0, 1)\}$ besteht (keine Rechnungen nötig)?

Aufgabe 2 — Anwendungen von morphologischen Operationen - 5+10+10=25 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.morphology`, `matplotlib`

Im Rahmen dieser Aufgabe sollen jeweils unterschiedliche Informationen aus den Bildern `roads_bin.png`, `zebra_bin.png` und `boxes_bin.png` mittels der geschickten Nutzung von morphologischen Operationen extrahiert werden.

1. Das Bild `roads_bin.png` stellt eine abstrakte Straßenkarte mit breiten Straßen und schmalen Straßen dar. Wendet eine oder mehrere geeignete morphologische Operationen sowie ein passendes Strukturelement an, um die schmalen Straßen aus dem Bild zu entfernen und die großen Straßen dabei nahezu komplett zu erhalten.
2. Das Bild `zebra_bin.png` zeigt die Form eines Zebras als binäres Bild. Ermittelt nun den Umfang des Zebras. Dieser sei vereinfacht definiert als die Summe der Pixel auf dessen Kontur. Nutzt dazu eine passende morphologische Operation mit einem passenden Strukturelement und verarbeitet das Ergebnis entsprechend weiter.
3. Im Bild `boxes_bin.png` sind mehrere Quadrate mit unterschiedlichen Seitenlängen dargestellt. Ermittelt mithilfe von morphologischen Operationen sowie einem passenden Strukturelement, wie viele Quadrate mit der Seitenlänge 10, 20, 40, 80 bzw. 160 es jeweils im Bild gibt. Um die Anzahl

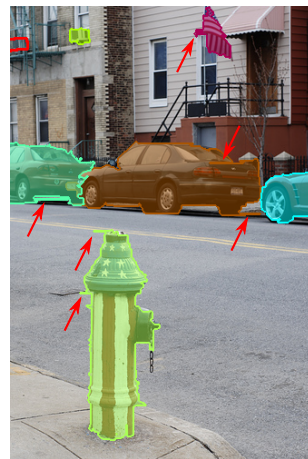
an Zusammenhangskomponenten im binären Bild zu zählen, etwa die Anzahl der Quadrate, könnt ihr folgende Python-Funktion nutzen:

```
from skimage.morphology import label
import numpy as np

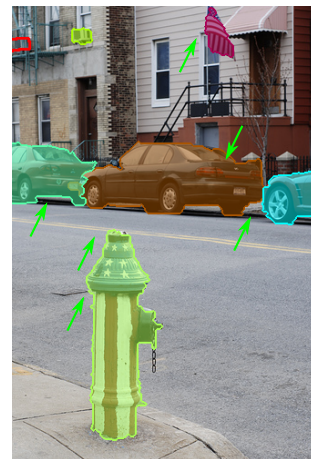
def countComponents(img):
    return np.max(label(img))
```



(a) Eingabebild



(b) Ergebnis ohne morphologische Nachverarbeitung



(c) Ergebnis mit morphologischer Nachverarbeitung

Abbildung 1: Eingabebild a und Ergebnisse der Objektentdeckung vor der morphologischen Nachverarbeitung b und nach der morphologischen Nachverarbeitung c. Die Antennen-artigen Artefakte in b sind mit roten Pfeilen markiert. Die selben Stellen sind in c mit grünen Pfeilen markiert.

Aufgabe 3 — Objektmasken verbessern - 20 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: numpy, skimage.io, skimage.morphology, matplotlib, glob

Die Abbildung 1b zeigt das Ergebnis eines Systems zur Entdeckung von Objekten auf dem Bild 1a bzw. 293071.jpg im Moodle. Insgesamt sind dort sechs Objekte gefunden wurden, wobei für jedes Objekt eine farbige Maske in das Bild eingezeichnet wurde. Für jede der farbigen Masken findet ihr ein entsprechendes binäres Bild im `objektmasken.zip`. Wie euch vielleicht auffällt, sind die Masken in Abbildung 1b bzw. dem `objektmasken.zip` nicht perfekt. Unter anderem existieren störende, dünne Antennen-artige Artefakte, die entlang horizontaler Linien aus den Masken hervorstechen oder aus diesen ausgeschnitten sind. Diese Artefakte sollen im Rahmen dieser Aufgabe entfernt werden. Ladet dazu die Masken als binäre Bilder und manipuliert sie mit geeigneten morphologischen Operatoren und Strukturelementen so, dass die Artefakte verschwinden und ein Ergebnis wie in Abbildung 1c entsteht. Dabei sollen die Bereiche, die nicht von den Artefakten betroffen werden, möglichst wenig verändert werden.

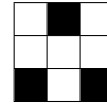
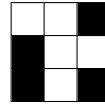
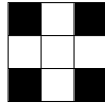
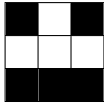
Hinweis: Zur Visualisierung eurer Masken könnt ihr die Funktion `visMasks(img, masks)` (s. `vis.ipynb` im Moodle) nutzen. Die Funktion erwartet das Bild als ersten Parameter und eine Liste an binären Bildern (Masken) als zweiten Parameter. Die Rückgabe ist dann die Visualisierung als RGB-Bild. Die Permutation der Farben der Masken ist unerheblich für das Ergebnis.

Aufgabe 4 — Verzweigungen zählen - 10+15=25 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: numpy, skimage.io, skimage.morphology, matplotlib

In dieser Aufgabe sollen im Binärbild `labyrinth_bin.png`, das ein Labyrinth mit Gängen der Breite 1 Pixel sowie Plätzen beliebiger Größe (weiß) und Wänden (schwarz) darstellt, alle Verzweigungen der Gänge gefunden werden, an denen drei oder vier Gänge horizontal oder vertikal zusammentreffen. Zudem sollen alle Zugänge zu den Plätzen, also Stellen, an denen ein Gang in einen Platz mündet, gefunden werden. Einige Beispiele für diese Verzweigungen zeigen die folgenden Abbildungen.

Beachtet, dass ihr nur die Kreuzungen in diesem Labyrinth finden müsst und nicht die Kreuzungen in einem beliebigen Labyrinth, etwa mit anderen Zugängen zu Plätzen.



1. Nutzt zur Lösung dieser Aufgabe die in der Vorlesung diskutierte Hit-or-miss Transform (HMT). Überlegt euch zunächst, wie die Strukturelemente aussehen müssen, welche die gesuchten Verzweigungen finden. Achtet darauf, dass die Verzweigungen in beliebige horizontale/vertikale Richtungen führen können.

Tipp: Vier Strukturelement sollten reichen.

2. Ladet nun das Bild `labyrinth_bin.png` und implementiert die HMT mit euren Strukturelementen. Geht bei der HMT davon aus, dass alle Pixel außerhalb des Bildes den Wert 0 haben. Markiert auf dem Bild, wo die gesuchten Verzweigungen liegen, indem ihr jeweils das Pixel im Zentrum der Verzweigung rot färbt.

Tipp: Es sollten 11 Verzweigungen gefunden werden.